

OCJP JAVA 17 & 21 PROGRAMMER CERTIFICATION FUNDAMENTALS

OFFICIAL COURSE COMPANION

CHAPTER 22

Date and Time API

CHAPTER OVERVIEW

When Java 8 introduced the new Date and Time API, it was Java's third attempt to get the date/time business correctly. The first one was the `java.util.Date` class and the second one was the `java.util.Calendar` class.

EXAM OBJECTIVES COVERED

- ◆ Manipulate date, time, duration, period, instant and time-zone objects including daylight saving time using Date-Time API

Chapter 22 Date and Time API

Exam Objectives

- Manipulate date, time, duration, period, instant and time-zone objects including daylight saving time using Date-Time API

22.1 Overview of the Date and Time API

When Java 8 introduced the new Date and Time API, it was Java's third attempt to get the date/time business correctly. The first one was the `java.util.Date` class and the second one was the `java.util.Calendar` class. Both of them were a pain to use and a cause of bugs in Java applications. Given that most non-trivial applications use data/time in some form or the other, you can imagine how frustrating it must have been to not have good abstractions for date/time values in the JDK. Yet, it took a long time for Java to come up with a better alternative. I only mention this to make you appreciate the fact that working with date/time is not as simple as it sounds. It is important to get the fundamentals right.

22.1.1 Important classes of the Date/Time API

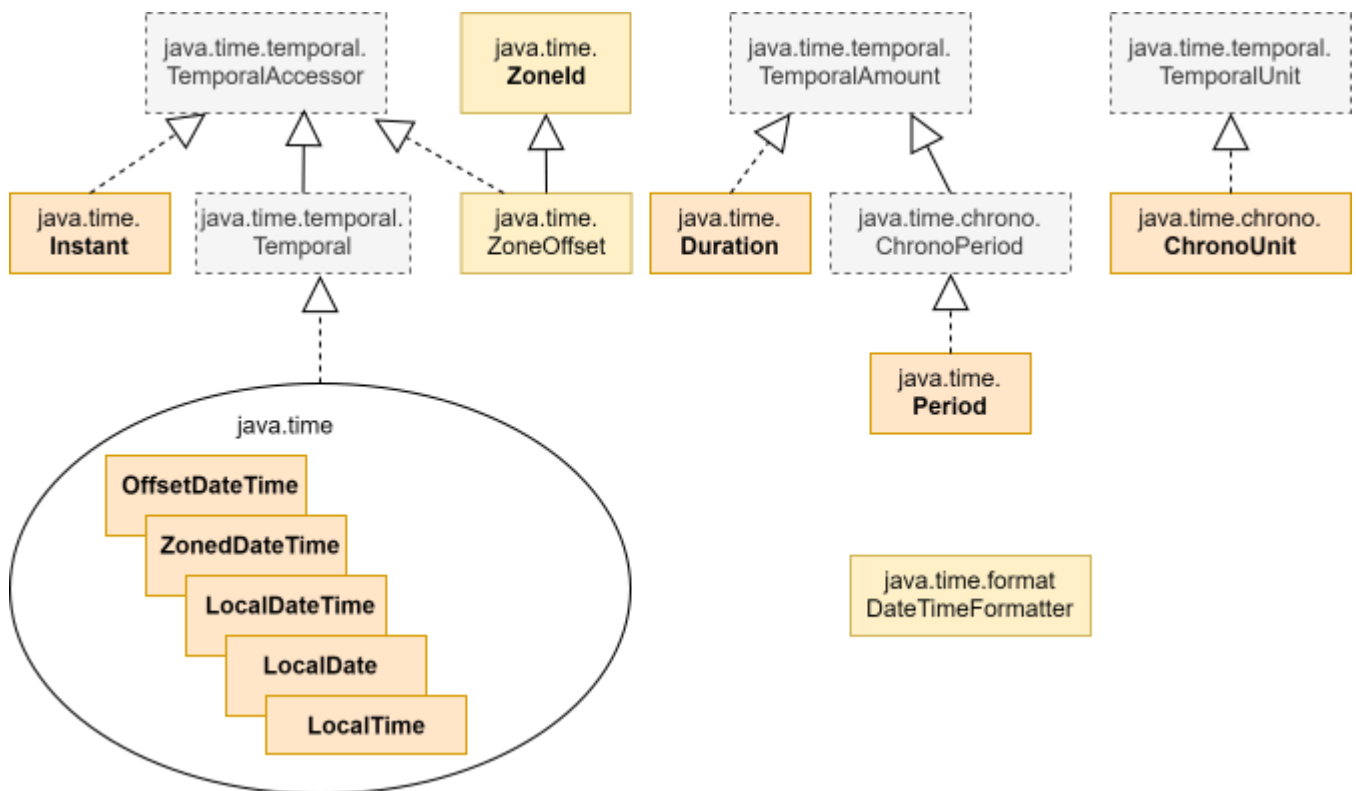
Java 8's Date/Time API is a completely fresh take on managing the date and time. It has no dependency and no relation to the old `Date` and `Calendar` classes. Although in real life you may still see code that uses the old `Date` and `Calendar` classes, I will not talk about them in detail here in order to avoid any confusion.

The main cause of complexity in working with time is the fact that machines don't see time the same way as humans do. To machines, time is just a period that has elapsed after a predefined starting point. This point in time is called "**Epoch**" and is set at

January 1, 1970 00:00:00.000 at Greenwich. (Greenwich is the name of the town in UK where the Royal Observatory is located. Time at this location was chosen for historical reasons). Time at this location is called Greenwich Mean Time (GMT). The same time has been adopted globally as the reference time and is also called Coordinated Universal Time or UTC for short. You can think of time as a series of milliseconds plotted on the x-axis, where 0 is called January 1, 1970 00:00:00.000 GMT. Every millisecond elapsed since the epoch moves you forward on this axis.

Humans, on the other hand, see time in terms of hours, minutes, and seconds of a specific day, month, and year, at a particular location. For example, to computers, the point 1735711200000 on the time line means 1735711200000 number of milliseconds have elapsed since the epoch. But to humans, this point is actually 6AM 1st January 2025 at **Greenwich**. The same point in time could also be interpreted as 1AM on January 1st 2025 in **New York** or 10PM on Dec 31st 2024 in **Los Angeles**. Old Java classes mix these two very different ideas in the same class. This is evident from the fact that `java.util.Date` actually just stores the number of milliseconds that have elapsed since epoch and has no idea about what date it is. Yet, the name of the class is **Date**!

The new Date/Time API addresses this concern well and makes it a lot easier to perform routine operations such as date arithmetic and printing. The new date/time framework is organized in the **java.time** package and has a lot of classes and interfaces. The following diagram shows **a few** important classes and interfaces of this API. Dashed boxes contain interfaces and regular boxes contain classes.



In the above diagram, ten classes are shown in bold. Even though the exam only focuses on these ten classes it is important to understand the framework of the related classes as well. Let's take an overview of the packages and classes shown in the above diagram first and then I will go over their details.

The `java.time` package 🙌

This is the most important package of the date/time framework. The majority of the application programs only need the classes defined in this package. The classes defined in this package are based on the ISO calendar system (You don't need to know the details of the ISO calendar system). All classes in this package are **immutable** and **thread-safe**.

The most important aspect of these classes is that there is a clear separation between the classes that represent the machine view of the time and the classes that represent the human view of the time.

Instant: This class represents the computer's view of time. As discussed before, a

computer only cares about the number of milliseconds that have elapsed since Epoch. That is exactly what an `Instant` contains. It has no notion of AM/PM or the time zone. However, since the zero of the computer's time line has been defined to correspond to 00.00AM 1st Jan, 1970 UTC, converting an `Instant` to the time in the UTC time line doesn't require any other information. Thus, you can say that an `Instant` object precisely corresponds to a date and time in UTC. You can create an `Instant` for any given amount of time elapsed since epoch. For example, `Instant.ofEpochMilli(1735711200000L)`.

LocalDate/LocalTime/LocalDateTime: These classes represent just a date, a time, or a date and time without any indication of the place for which that date/time is applicable. Think of it this way - if someone tells you that the President will address the nation at 8PM on 1st Jan 2025, you can't really be sure exactly when you need to be ready to watch it because you haven't been told the place for which this date and time is applicable. Without this information you may be too early or too late to watch the speech. Thus, you can't take the information available in a `LocalDateTime` object and identify a specific point on the machine time line. You can, at best, guess that this time is with respect to the region you are in and use that information to decide when to be ready to watch the President's address. In that sense, it represents the date/time of the system on which the program is running. You will see several ways to create objects of these types in the next section.

ZoneId: A `ZoneId` identifies an area or a region, known as "Time Zone", for which precise rules for determining the date and time of a day with respect to the UTC exist. These rules enable you to take a point on the machine's timeline, i.e., an `Instant` object, and determine the exact date and time in that region at that instant. Note that a `ZoneId` object itself doesn't contain any rule. It is just an id. JDK maintains the rules in an internal database of sorts (in `ZoneRules` class, actually, but you don't need to worry about it) within its standard library, which can be retrieved using this id. There are many ways to create a `ZoneId` object but usually, you will see a `ZoneId` being created using a string of the form `<area>/<city>`. For example, `ZoneId.of("Asia/Calcutta")` or `ZoneId.of("America/New_York")`. If you check out the JavaDoc of the `of` method, you will notice that the underlying framework provided by the JDK is quite robust and can create the `ZoneId` from a

string written in many different formats. Even so, the string used to create a `ZoneId` using the `of` method must either match an official time zone in the JDK's database or be in a format that correctly specifies an "offset" amount such as UTC+01:15. (Don't worry, the format is not important for the exam). Trying to create a `ZoneId` for a non-existing place such as `ZoneId.of("Somewhere/Timbaktoo");` will cause a `java.time.zone.ZoneRulesException` to be thrown with a message, Unknown time-zone ID: Somewhere/Timbaktoo.

ZonedDateTime: A `ZonedDateTime` object represents a date and time in a particular time zone. Thus, it needs two things - the number of milliseconds elapsed since the epoch and the `ZoneId` of the time zone whose time it represents. It uses the `ZoneId` to fetch all the rules that affect the date/time used in that zone and applies them to show you the regional time. In layman terms, for example, if people in a particular zone shift their clocks backward or forward on certain dates in a year (such as for saving daylight), then a `ZonedDateTime` object created for that zone knows those dates and the amount of shift that needs to be applied to show the correct time in that region at any instant. Since a `ZonedDateTime` pulls all applicable rules for that zone using the `ZoneId`, a `ZoneId` is necessary to create a `ZonedDateTime`.

OffsetDateTime: - The `OffsetDateTime` class is a bit weird because it neither represents a local datetime nor does it represent a datetime in any particular geographical region. It actually contains just two things - the number of milliseconds elapsed after the epoch and an offset amount called "zone offset", which is nothing but the amount by which time represented by this `OffsetDateTime` differs from UTC. Since an `OffsetDateTime` does not represent time for any specific geographical region, it does not care about any region specific rules. In a way, an `OffsetDateTime` object represents a date/time of an imaginary place that is always "offset" from the UTC by the given offset value. The offset amount is captured by an instance of the `ZoneOffset` class, which **extends** `ZoneId`. You can actually create a zone offset of any amount like this:

```
ZoneOffset zo = ZoneOffset.ofHoursMinutes(1, 10); //creates a ZoneOffset of 1 hour and 10 minutes
```

The "id" of this `ZoneOffset` is `"+01:10"`. Of course, you won't be able to find any "rules" for this zone because `+1:10` is not an official time zone name of any region in

the world but if you create a `ZonedDateTime` using this `ZoneOffset`, the time in this imaginary place will always be 1 hour and 10 minutes ahead of UTC.

Period: Period represents a date-based amount of time in terms of days, months, and years. It has no notion of hours, minutes, and seconds.

Duration: Duration represents time-based amount of time in terms of Hours, Minutes, and Seconds. It has no notion of days, months, or years.

It is important to understand the difference between **Period** and **Duration**. Even though a Period representing 1 day is the same as a Duration representing 24 hours, both are conceptually different. Adding a Period of 1 day to a given date will only increment the day part of the date and will not affect the time component but adding a Duration of 24 hours will add 24 hours to the time part, which in turn, changes the day part as well. For example, adding a Period of 1 Day to the date before **day light savings start date**, will simply increment the day without affecting the time. Thus, 9th March 2024 10 PM will become 10th March 2024 **10 PM**. But adding a Duration of 24 hours to the same date will change the date to 10th March 2024 **11 PM**, because on this particular day, clocks were moved forward by 1 hour. Therefore, 10PM + 24 hours is not 10PM but 11PM of the next day.

This package also contains two enums - `Month` and `DayOfWeek`. `Month` contains values from `JANUARY` to `DECEMBER` and `DayOfWeek` contains values from `MONDAY` to `SUNDAY`. It is better to use these enums instead of integers while creating date/time objects because they make the code more readable and they also let you avoid the confusion caused by indexing. For example, `LocalDate.of(2024, Month.JANUARY, 10)` is more readable than `LocalDate.of(2024, 1, 10)`.

The `java.time.temporal` package 🙌

This package contains the base interfaces such as `Temporal`, `TemporalAmount`, `TemporalUnit`, `TemporalField` and `TemporalAccessor` of the Date/Time framework. These interfaces are implemented by various classes of the **java.time** package.

Having common base interfaces for related classes allows applications to operate on the classes uniformly. For example, you can add a `Period` to a `LocalDate` in the same way as you add a `Duration` to a `ZonedDateTime`, i.e., by calling the `addTo` method on a `Period` or a `Duration` instance or by calling the `plus` method on a `LocalDate` or a `ZonedDateTime` instance. This is possible because both `Period` and `Duration` implement `TemporalAmount`, which declares the `addTo(Temporal t)` method and both `LocalDate` and `ZonedDateTime` implement `Temporal`.

The exam does not mention this package in the objectives but the whole scheme of things related to date/time manipulation is driven by the interfaces of this package. Many of the methods that you will use while working with date/time classes are actually declared in the interfaces of this package.

The `ChronoUnit` enum of this package is particularly useful as it implements `TemporalUnit` and defines a standard set of units of date/time such as `DAYS`, `MONTHS`, `YEARS`, `HOURS`, `MINUTES`, `SECONDS`, `NANOS`, and `WEEKS`. These units are used while manipulating and creating date and time objects. `ChronoUnit` also overrides the `long between(Temporal temporal1Inclusive, Temporal temporal2Exclusive)` method of `TemporalUnit`, which is used often to calculate the difference between two date/time values in terms of the unit defined by the `ChronoUnit` instance.

The `java.time.format` package 🙌

This package contains helper classes for converting date/time objects to strings and for parsing strings into date/time classes. **DateTimeFormatter** and **FormatStyle** are the most used classes of this package.

If you browse through the JavaDoc of the new Date/Time API, you will see many different classes and many different methods in each of these classes. It is easy to get overwhelmed, but don't worry. The API is actually quite easy to understand once you realize that there are only four types of operations that you can do with these classes - creating new objects, comparing, reading the state of the objects, and converting them to strings. There are no methods for updating because the objects of these classes are immutable. All relevant classes have similar methods to perform these operations. In the following sections, I will explain each of these operations with examples.

22.2 Creating Date/time Objects 🙌

Remember that none of the date/time classes have a public constructor. Therefore, it is not possible to create them using the `new` operator. If you see something like `new LocalDate()` anywhere in the code, go straight for the option that says, "compilation fails"!

Instead of having constructors, the new date/time API provides several other ways to create date/time objects. They can broadly be classified into two categories:

1. The overloaded **public static** factory methods named `now`, `of`, and `from`
2. The overloaded **public instance** methods named `with` and `truncatedTo`

Let's check them out.

22.2.1 Using the static now/of/from methods 🙌

The `now` methods 🙌

The **now** methods returns an instance that represents the current date or time in the time line that the class is meant to represent. For example, since the `LocalDateTime` class represents the date and time of the system on which the program is running, `LocalDateTime.now();` returns a `LocalDateTime` object set

to the current date and current time as shown by the system's clock. Remember that `LocalDateTime` does not keep any time zone information. So, `LocalDateTime.now()` just picks up the current date and time values from the local machine and ignores the time zone. `LocalDate.now()` and `LocalTime.now()` work similarly.

`OffsetDateTime.now()` and `ZonedDateTime.now()` do the same thing except that they also pick up the system clock's offset and time zone information respectively.

The `Instant` class, on the other hand, represents the number of milliseconds elapsed since epoch (remember?) and so, `Instant.now()` returns the an `Instant` object that contains the number of milliseconds elapsed since epoch as of the moment that `Instant` object is created.

The following code, that I ran on 26th Apr 2024 8:30:42 PM in **India** (my system's time zone is set to Indian Standard Time aka **IST**, which is 5.30 hours ahead of **UTC**), makes this clear:

```
out.println(LocalDate.now()); //prints 2024-04-26T20:30:42.761568900
out.println(OffsetDateTime.now()); //prints 2024-04-
26T20:30:42.762565600+05:30
out.println(ZonedDateTime.now()); //prints 2024-04-
26T20:30:42.762565600+05:30[Asia/Calcutta]
out.println(Instant.now()); //prints 2024-04-26T15:00:42.762565600Z
```

The output generated by the above code shows that `LocalDateTime` doesn't read any offset or time zone information from the system's clock. `OffsetDateTime` reads the offset information but does not read the time zone information. `ZonedDateTime` reads both.

Observe the letter `z` that is appended to the output of `Instant.now()`. It implies **UTC**. Also observe that the time printed by `Instant.now()` is 5.30 hours before

local, offset, and zoned datetimes. That's because, **IST** is 5.30 hours ahead of **UTC**.

There are other overloaded variants of the `now` method that work on the same principle. For example, you may create a `ZonedDateTime` that represents the current time in another time zone using `ZonedDateTime.now(ZoneId zid)`. So, if I call `ZonedDateTime.now(ZoneId.of("America/New_York"))` when my system time shows 26th Apr 2024 9PM IST, it will return the date and time representing 26th Apr 2024 11.30AM EST because US/Eastern time zone is 9.30 hours behind IST. (IST is 5.30 hours ahead of UTC and EST is 4 hours behind UTC, so, EST is 9.30 hours behind IST)

Since `Period` and `Duration` denote an amount of time and not a point in time, they don't have the `now()` method. But they do have a static constant named `ZERO`, which denotes a period/duration of zero amount.

The `of` methods 🙋

The `of` methods lets you create an object for any date or time by passing individual components of a date/time. There are too many methods of this type to list here, so, I will just present a few examples to illustrate how they are used:

```
LocalDate ld = LocalDate.of(2024, 1, 10); //10th January 2024
LocalTime someTime = LocalTime.of(10, 53); //10.53 AM
LocalDateTime ldt1 = LocalDateTime.of(2024, 1, 10, 23, 10, 14);
OffsetDateTime.of(LocalDateTime.now(), ZoneOffset.ofHours(5));
OffsetDateTime odt2 = OffsetDateTime.of(2024, 1, 1, 10, 45, 50, 0,
ZoneOffset.ofHours(5));
ZonedDateTime zdt2 = ZonedDateTime.ofInstant(Instant.now(),
ZoneId.systemDefault());
```

There are three points that you should remember about the `of` methods:

1. Indexing for month parameter starts from `1`. In other words, to specify **January**, you must pass `1` (or use the enum value `Month.JANUARY`).
2. They use a **24 hour clock** for the `hour` parameter. Thus, to specify **11 AM**, you need to pass `11` and to specify **11 PM**, you need to pass `23`.
3. They throw `java.time.DateTimeException` (which extends `java.lang.RuntimeException` and is, therefore, an unchecked exception) if the value passed for any field is out of range. For example, `LocalDate.of(2018, 2, 29);` will throw a `DateTimeException` because February of 2018 doesn't have 29 days. Similarly, `LocalTime.of(24, 10);` will also throw a `DateTimeException` because the range of the hour parameter is only 0 to 23.

The `Period` class has the following `of` methods:

```
static Period of(int years, int months, int days)
static Period ofDays(int days)
static Period ofMonths(int months)
static Period ofWeeks(int weeks)
static Period ofYears(int years)
```

Here are a few examples that show how they can be used:

```
out.print(Period.ofDays(40)); //prints P40D
out.print(Period.ofMonths(-2)); //negative values are also acceptable. prints P-2M
out.print(Period.of(0, 1, 10)); //prints P1M10D
```

Note that a `Period` of 40 days is not the same as a `Period` of 1 month and 10 days because 1 month is not always equal to 30 days.

The `Duration` class has similar `of` methods:

```
System.out.println(Duration.ofDays(1)); //prints PT24H
System.out.println(Duration.ofHours(24)); //prints PT24H
System.out.println(Duration.of(24, ChronoUnit.HOURS)); //prints
```

The `from` methods 🙌

The `from` methods are similar to the `of` methods except that instead of specifying a value for each property individually, you specify another `TemporalAccessor` object. This `TemporalAccessor` object is used as a source for all properties that are required to create the desired object. For example:

```
LocalDateTime ldt = LocalDateTime.now();
LocalDate ld1 = LocalDate.from(ldt);
LocalTime lt = LocalTime.from(ldt);
OffsetDateTime odt1 = OffsetDateTime.from(ZonedDateTime.now());
```

Observe that a `LocalDateTime` has a date as well as a time component and therefore, it is possible to use a `LocalDateTime` to create a `LocalDate` as well as a `LocalTime`. But it is not possible to use a `LocalDate` to create a `LocalDateTime` or a `LocalTime` because `LocalDate` does not have a time component. Similarly, you can't create a `ZonedDateTime` using a `LocalDateTime` because `LocalDateTime` doesn't have zone information. Thus, the following code will throw a `DateTimeException` at run time:

```
LocalDate ld = LocalDate.now();
LocalDateTime ldt = LocalDateTime.from(ld); //DateTimeException at run
time
LocalTime lt = LocalTime.from(ld); //DateTimeException at run time
ZonedDateTime zdt = ZonedDateTime.from(LocalDateTime.now());
//DateTimeException at run time
```

Similarly, you can create a `Period` using another `Period` but you can't create a `Period` using a `Duration` because `Duration` deals with hours, minutes, and

seconds, while `Period` needs days, months, and years. Creating a `Duration` from a `Period` won't work either.

22.2.2 Using the `plus/minus/with/at/truncatedTo` instance methods

Remember that all date/time objects are immutable and therefore, you cannot change them. But you can use the values from an existing date/time to build a new date/time object. This is helpful when the new object that you want to create is slightly different from an existing object. For example, you may already have a `LocalDate` and you want to create a `LocalDateTime` by adding a time component to the same date, or you have a `LocalDateTime` and you want just a `LocalDate`, or you have a `LocalDate` for today and you want a `LocalDate` for tomorrow. The API provides several methods for this purpose.

To make them easy to remember, I have categorized these methods on the basis of how they alter the existing values:

1. methods that return a new object by altering a field of another object - `plus/minus/with/truncatedTo`
2. methods that return a new object by adding or removing a field to an another object - `at/truncatedTo`

1. Instance methods that alter an existing field value

The `plus/minusXXX` methods

Both - a `LocalDate` and a `Period` - have three components: day, month, and year. To modify any of these properties of, you have `plusDays`, `plusWeeks`, `plusMonths`, and `plusYears` and `minusDays`, `minusWeeks`, `minusMonths`, and

`minusYears` methods in `LocalDate` as well as in `Period`. Similarly, `LocalTime` has `plusHours`, `plusMinutes`, `plusSeconds`, `plusNanos` and `minusHours`, `minusMinutes`, `minusSeconds`, `minusNanos` methods to modify the properties that it supports.

Note that `LocalTime` does not have methods to alter the day, month, and year because `LocalTime` does not have these properties, while `LocalDateTime` has plus and minus methods for all properties of `LocalDate` as well as `LocalTime`.

Example:

```
LocalDate ld1 = LocalDate.now();  
LocalDate ld2 = ld1.plusDays(10);  
Period p = Period.ZERO.plusDays(10);
```

The **plusXXX** and **minusXXX** methods take only one argument because the name of the property to be updated is implied by their names.

There are four more `plus` and `minus` methods that do the same thing but in a different way - `plus(long amountToAdd, TemporalUnit unit)`, `plus(TemporalAmount amountToAdd)`, `minus(long amountToSubtract, TemporalUnit unit)`, `minus(TemporalAmount amountToSubtract)`. These methods identify the property to be altered through the arguments that are passed to them.

Example:

```
LocalDate ld1 = LocalDate.now();  
LocalDate ld2 = ld1.plus(10, ChronoUnit.DAYS);//add 10 days  
LocalDate ld3 = ld1.minus(Period.of(0, 0, 10));//subtract 0 yrs, 0 months, 10 days  
LocalDate ld4 = ld1.plus(Period.ofDays(-10));//add -10 days,
```

i.e., subtract 10 days

Remember from the class hierarchy diagram that I showed earlier that `Period` implements `TemporalAmount` and `ChronoUnit` is an enum that implements `TemporalUnit`.

You need to be careful about using days and months while manipulating dates. Adding a month to a date is not the same as adding 30 days to the date because the number of days in a month depends on the month of the date that you want to manipulate. The following example shows the difference:

```
LocalDate ld = LocalDate.of(2024, 01, 31);
LocalDate ld1 = ld.plusDays(30);
System.out.println(ld1); //prints 2024-03-02
LocalDate ld2 = ld.plus(Period.ofMonths(1));
System.out.println(ld2); //prints 2024-02-29
```

If you try to add to or subtract from a field that is not supported by a particular class, a `DateTimeException` will be thrown. For example, adding hours to a `LocalDate` or adding months to a `LocalTime`, will throw a `DateTimeException` at runtime.

The `withXXX` methods

If you want to specify an exact value for a property instead of adding to or subtracting from a property, you can use the `withXXX` methods.

Example:

```
LocalDate ld = LocalDate.now(); //doesn't matter what the year is now
LocalDate ld2 = ld.withYear(2025); //change year to 2025
ld2 = ld.withMonth(2); //change month to Feb
ld2 = ld.withDayOfMonth(4); //change date to 4

LocalDateTime ldt = LocalDateTime.now();
```



```
ldt = ldt.withYear(2019); //change year to 2019
ldt = ldt.withHour(23); //change hour to 23
```

`Period` has similar **withXXX** methods exception that the names of the methods end with an 's':

```
Period p1 = Period.ZERO.withDays(10);
p1 = p1.withMonths(2);
p1 = p1.withYears(1); //p1 now points to a Period with 1 year, 2
months, and 10 days
```

`Period` also has a **negated** method that returns a new `Period` with amount for each component negated. For example `Period.of(1, 3, 20).negated();` returns a `Period` of -1 year, -3 months, and -20 days. `Duration` has similar `with` and `negated` methods.

2. Instance methods that add new properties or remove properties to/from an existing object - `at` and `truncatedTo`

The **atXXX** methods

What if you have a `LocalDate` and you want a `LocalDateTime` by adding a time component to the existing date? Well, `LocalDate` has several `atTime` methods that let you create a `LocalDateTime` using a `LocalDate`.

Example:

```
LocalDate ld = LocalDate.now();
LocalDateTime ldt = ld.atTime(10, 15); //same date with 10hr 15mins
ldt = ld.atTime(10, 15, 45); //same date with 10hr 15mins 45 secs.
```

Similarly, `LocalTime` has `atDate(LocalDate ld)`, which returns a `LocalDateTime` and `LocalDateTime` has `atZone(ZoneId id)`, which returns a

`ZonedDateTime` .

```
LocalTime lt = LocalTime.now();
LocalDate ld = LocalDate.now();
LocalDateTime ldt = lt.atDate(ld); //add a LocalDate to LocalTime.
```

Note that `LocalDateTime` does not have `atTime` or `atDate` because `LocalDateTime` already has a time as well as a date component. But it does not have an offset or a zone. So, `LocalDateTime` does have `atOffset` and `atZone` methods:

```
ZonedDateTime zdt =
LocalDateTime.now().atZone(ZoneId.of("America/New_York"));
```

`Period` and `Duration` do not have any **atXXX** method.

The `truncatedTo` method

The `truncatedTo` method returns a copy of the original date-time with fields smaller than the specified unit set to zero. For example, truncating with the `DAYS` unit will set the hours, minutes, seconds, and nano fields to zero:

```
ZonedDateTime zdt1 = ZonedDateTime.ofInstant(Instant.now(),
ZoneId.systemDefault());System.out.println(zdt1.truncatedTo(ChronoUnit.DAYS));
//prints 2024-04-27T00:00+05:30[Asia/Calcutta]
```

Although not important for the exam, be aware that not all date-time classes support truncation with all available units. For example, `ZonedDateTime` doesn't support truncation with units greater than `DAYS` .

22.2.3 Creating a Date/Time object using the static parse methods

All date/time classes have two parse methods that return an object of the respective class using information present in the string argument. For example, `LocalDate` has the following two methods:

```
static LocalDate parse(CharSequence text)
```

```
static LocalDate parse(CharSequence text, DateTimeFormatter formatter)
```

These are quite straight forward to use:

```
LocalDate ld = LocalDate.parse("2018-02-14");  
LocalTime lt = LocalTime.parse("22:10:30");  
LocalDateTime ldt = LocalDateTime.parse("2018-02-14T18:12:12");
```

The way these methods parse a given string needs a little explanation. They can't just parse any random string into a date/time. The string that you pass must contain date/time data in a specific format. This format is described by an object of class `java.time.format.DateTimeFormatter`.

A `DateTimeFormatter` object understands the format in which date/time information is present in a string. While it is possible to design your own format for writing a date, standard formats for writing the date/time strings exist, and `DateTimeFormatter` class provides several readymade `DateTimeFormatter` objects that understand these standard formats. These object can be accessed through public static variables of `DateTimeFormatter` class. You need to be aware of three such variables - `ISO_LOCAL_DATE`, `ISO_LOCAL_TIME`, and `ISO_LOCAL_DATE_TIME` - because when you don't pass a formatter to the `LocalDate/Time`'s parse methods, these are the formatters that the parse methods use internally to parse the string. `LocalDate` uses `ISO_LOCAL_DATE`, `LocalTime` uses `ISO_LOCAL_TIME`, and `LocalDateTime` uses `ISO_LOCAL_DATE_TIME` to parse the given string. Thus, for example, invoking `LocalDate.parse("2024-02-14");` will produce the same result as invoking `LocalDate.parse("2024-02-14", DateTimeFormatter.ISO_LOCAL_DATE);`

For the purpose of the exam, you only need to know that `ISO_LOCAL_DATE` expects strings in `yyyy-MM-dd` format, `ISO_LOCAL_TIME` expects strings in `HH:mm` or `HH:mm:ss` format, and `ISO_LOCAL_DATE_TIME` expects strings in `yyyy-MM-ddTHH:mm:ss` or `yyyy-MM-ddTHH:mm` format (ignoring the nano second part of the time component), where `yyyy` stands for four digit year, `MM` stands for two digit month, `dd` stands for two digit date, `HH` stands for two digit hour, `mm` stands for two digit minute, and `ss` stands for two digit second. You will not be required to parse complicated strings in the exam.

Furthermore, unless the problem statement says otherwise, you should answer the question assuming that the code is executed with **English/US** as its language. Therefore, the names and short codes of the months would be in English.

Similarly, `OffsetDateTime`'s `parse` methods use `DateTimeFormatter.ISO_OFFSET_DATE_TIME` by default, `ZonedDateTime`'s `parse` methods use `DateTimeFormatter.ISO_ZONED_DATE_TIME` and so on. You do not need to know the formatting details of these formatters, but you do need to know that they exist so that you won't assume an option to be incorrect just because it uses one of these formatters.

The `DateTimeFormatter` class 📌

Creating a `DateTimeFormatter` to parse a non-standard string is also quite easy. `DateTimeFormatter` has a static `ofPattern(String pattern)` and a static `ofPattern(String pattern, String locale)` method that take the format specification as an argument. For example, if your dates are in **MMM/dd/yy** format, you can create a formatter like this:

```
DateTimeFormatter dtf = DateTimeFormatter.ofPattern("MMM/dd/yy");
```

```
LocalDate ld = LocalDate.parse("Oct/23/19", dtf);//assuming English/US  
System.out.println(ld);//prints 2019-10-23
```

Format specification uses predefined characters to denote various components of a date/time. While a detailed understanding of this specification is not required, you should be aware of the meaning of a few basic letters:

- y is for year
- M is for month
- d is for day of month
- H is for hour of day
- m is for minute
- s is for second
- n is for nanosecond

Parsing a string to create a **Period** or a **Duration** 🙌

The **Period** also has a parse method but the format of a period string is quite different from date/time. It looks weird to be honest when you look at it for the first time but it actually makes sense. It follows ISO-8601 period format and looks like this: **PnYnMnD** and **PnW**. Here, **n** stands for a number and **P**, **Y**, **M**, **D**, and, **W** are letters that appear in the string. The case of the letters is not important. For example, **P1Y10M3d** means 1 year, 10 months, and 3 days, **P1Y3D** means 1 year and 3 days. **p1Y2w** means 1 year and 2 weeks.

It is now easy to create a **Period** by parsing a string. For example, **Period p = Period.parse("p1Y2w");** creates a **Period** with 1 year and 2 weeks. You can create a **Period** with negative values as well, for example, **"-P2m3d"** and **"P-2m-3d"** will be parsed into the same **Period** representing -2 months and -3 days.

Parsing a **Duration** works similarly but it uses the character **T** to isolate the time information in the input string:

```
System.out.println(Duration.parse("P1DT3H4M"));
```

```
//prints PT27H4M
```

Observe that since `Duration` considers 1 day as 24 hours, it prints 27H in the above output.

You will **not** be asked to parse a complicated string in the exam. The exam does not have trick questions in this area and the basic knowledge given above will be sufficient for you to answer the questions in the exam.

22.3 Accessing Date/time objects

22.3.1 Reading the state of date/time objects 🙌

All date/time objects allow reading of their properties through appropriate getters. For example, JavaDoc API description for `LocalDateTime` lists the following getters:

`int getDayOfMonth()` : Gets the day-of-month field.

`DayOfWeek getDayOfWeek()` : Gets the day-of-week field, which is an enum `DayOfWeek`.

`int getDayOfYear()` : Gets the day-of-year field.

`int getHour()` : Gets the hour-of-day field.

`int getMinute()` : Gets the minute-of-hour field.

`Month getMonth()` : Gets the month-of-year field using the `Month` enum.

`int getMonthValue()` : Gets the month-of-year field from 1 to 12.

`int getNano()` : Gets the nano-of-second field.

`int getSecond()` : Gets the second-of-minute field.

`int getYear()` : Gets the year field.

Besides the above, it also has the following two special `get` methods:

`int get(TemporalField field)` : Gets the value of the specified field from this date-time as an int.

`long getLong(TemporalField field)` : Gets the value of the specified field from this date-time as a long.

These methods can be used like this:

```
System.out.println(LocalDate.now().get(ChronoField.DAY_OF_MONTH));
```

`LocalDate` and `LocalTime` have the same get methods except for the properties that they don't have. For example, `LocalDate` doesn't have `getHour` and `LocalTime` doesn't have `getYear`.

Properties supported by `Period`

`Period` has the following getters:

`int getDays()` : Gets the amount of days of this period.

`int getMonths()` : Gets the amount of months of this period.

`int getYears()` : Gets the amount of years of this period.

`long get(TemporalUnit unit)` : Gets the value of the requested unit.

Note that I have merely reproduced the descriptions from the JavaDoc because there is not much to discuss about them. They work as indicated by their names. The exam does not have trick questions on this aspect either.

Properties supported by `Duration`

Since `Duration` models a quantity or amount of time in terms of seconds and nanoseconds, `Duration` does not have `getHours()` or `getMinutes()`. It only supports `getSeconds()`, `getNano()`, and `get(TemporalUnit unit)` methods.

22.3.2 Chaining method calls 🙌

Most of the methods that you have seen thus far for creating new date/time objects (i.e. `now`, `of`, `ofXXX`, `from`, `plusXXX`, `minusXXX` and `withXXX`) return a reference of the same type. That makes it easy for you to customize the object you want by chaining multiple method calls in one statement. For example:

```
LocalDate ld = LocalDate.now().withYear(2010).plusMonths(2).minusDays(10);
```

First, you should get used to evaluating the result of such chained method calls because you will see such code on the exam. Second, evaluating such method calls is usually straight forward but you need to watch out for the following tricks that you will encounter in the exam:

1. **Watch out for lost objects** - I have said it multiple times already but since it is very easy to forget, I will say it again - all date/time objects are immutable. Therefore, none of the methods change the object they are invoked on. They return a new object. If you don't assign this new object to a variable, you won't see the result of your method calls. If you can tell what the following code will print, you know what I am talking about:

```
LocalDate ld = LocalDate.of(2018, 1, 1);  
ld.plusMonths(2).plusDays(10);  
System.out.println(ld);
```

You should change the second line to `ld = ld.plusMonths(2).plusDays(10);` if you want to see `2018-03-11` in the output.

2. **Watch out for return types** - Unlike the other creation methods, the **atXXX** methods return a reference of a different type. For example, `LocalDate`'s `atTime` method returns a `LocalDateTime`. If a call to this method is

2. embedded in the call chain, pay close attention to the type of the reference variable to which the result is being assigned. For example, the following code will not compile:

```
LocalDate ld = LocalDate.of(2018, 1, 1);  
ld = ld.withYear(2010).plusDays(10).atTime(10, 20).plusMonths(2);
```

Observe that the `of` method of `LocalDate` is being invoked above and therefore, it returns a `LocalDate`. For the same reason, the call to `plusDays` also returns a `LocalDate`. But the call to `atTime` on a `LocalDate` returns a `LocalDateTime` and therefore, the next call to `plusMonths` in the chain returns a `LocalDateTime`. A `LocalDateTime` object cannot be assigned to a variable of class `LocalDate`.

3. **Watch out for unsupported properties** - You can't add days to a `LocalTime` or hours to a `LocalDate`. Thus, while you may be led to believe that the following code will print `"2018-01-01"` (if you manage to avoid the first trap) but the fact is that it will not compile because `LocalDate` doesn't even have methods that alter time related fields.

```
LocalDate ld = LocalDate.of(2018, 1, 1);  
ld.plusHours(10); // will not compile  
System.out.println(ld);
```

4. **Watch out for chaining of static methods of Period** - This one is better explained through code. What do you think the following code will print?

```
LocalDate ld = LocalDate.of(2018, 1, 31).of(2019, 1, 1);  
System.out.print(ld+" ");  
Period p1 = Period.ZERO;  
Period p2 = p1.ofDays(1).ofMonths(1);  
System.out.println(p2);
```

4. It prints - `2019-01-01 P1M` . From the output, it looks like the call to `ofDays(1)` on `p1` had no effect!

Actually, the call to `of(2018, 1, 31)` on `LocalDate` had no effect either but the result produced in case of a `Period` is more alarming.

In both the cases, the **ofXXX** methods are **static** methods and produce a new object from scratch. Unlike the instance methods (i.e. with, plus, minus,), the static methods (of and parse) don't build "on top of" a previous object. As you know, it is technically valid to invoke a static method on a reference variable but invoking a static method on a reference is the same as invoking a static method on the class name. Thus, `LocalDate.of(2018, 1, 31)` produces a new `LocalDate` instance and chaining another `of` method to this `LocalDate` instance is actually the same as invoking `LocalDate.of(2019, 1, 1)` independently. However, since both the calls provide values for all the fields of a `LocalDate` , there is no difference in the end result.

Similarly, in case of `Period` , invoking `ofMonths(1)` on an existing `Period` instance is the same as invoking `Period.ofMonths(1)` . However, since `ofMonths` method creates a `Period` with 0 years and 0 days, it seems as if this call lost the value of the days property that was set earlier through `Period.ofDays(1)` .

In fact, none of the static methods make use of the values of an existing date/time instance even if you invoke the static method using a reference instead of the class name. If you want to create a `Period` with 1 month and 1 day, you should use `Period.of(0, 1, 1);` .

22.4 Translating Date/time objects into different types and different time zones 🙌

As explained earlier, humans cannot comprehend what "1735711200000 milliseconds since epoch" means. For example, you cannot tell someone to call you at

"1735711200000 since epoch" even though it is a perfectly valid specification of the time when you want them to call. Humans need a format such as 1st Jan 2025 6AM UTC to make sense of time. However, even a time written in this format can be inconvenient when people are spread out geographically and are in different time zones. For example, if you are in New York, you may be befuddled if you are asked to dial in for a team meeting at 2PM UTC on 29th March 2024! You would expect to see the time in your time zone, i.e., 10AM EST. Of course, your remote colleague in India may not be too happy to see that they have to dial in at 7.30PM but that's a different issue!

Java's Date/Time API makes it quite easy to do these translations. Let's start with the example discussed above. One way to capture a date/time at a zone is to create a `ZonedDateTime` using `LocalDateTime` and the time zone that you want. Like this:

```
LocalDateTime ldt = LocalDateTime.of(2024, 04, 29, 10, 0, 0); //no time zone here  
ZonedDateTime nyTime = ldt.atZone(ZoneId.of("America/New_York"));  
//time in New York, 2024-04-29T10:00-04:00[America/New_York]
```

To see the same time in another time zone, you can do:

```
Instant ins = nyTime.toInstant(); //get the time on machine's time line, i.e., UTC  
ZonedDateTime indiaTime = ins.atZone(ZoneId.of("Asia/Calcutta"));  
//translate it to the desired time zone
```

Or you can use the `ZonedDateTime`'s `withZoneSameInstant` method to do it in one call:

```
ZonedDateTime indiaTime =  
nyTime.withZoneSameInstant(ZoneId.of("Asia/Calcutta"));
```

Both the approaches will set `indiaTime` to 2024-04-

29T19:30+05:30[Asia/Calcutta].

Translating between `ZonedDateTime` and `OffsetDateTime`

Since `ZonedDateTime` and `OffsetDateTime` both contain a zone id (remember that `ZoneOffset` extends `ZoneId`), it is possible to translate either of these two types to the other. However, recall that the `ZoneId` of an `OffsetDateTime` does not really correspond to an actual region. It just contains the offset amount (such as `"-4:00"`). Translating a `ZonedDateTime` to an `OffsetDateTime` will lose the region information while keeping the offset amount. Therefore, converting that `OffsetDateTime` back to the **same** `ZonedDateTime` will not be possible as shown by the following code:

```
ZoneId nyZone = ZoneId.of("America/New_York");
ZonedDateTime nyZdt = ZonedDateTime.of(LocalDate.of(2024, 5, 1, 6, 0,
0), nyZone);
System.out.println(nyZdt); //prints 2024-05-01T06:00-
04:00[America/New_York]
OffsetDateTime nyOdt = nyZdt.toOffsetDateTime();
System.out.println(nyOdt); //prints 2024-05-01T06:00-04:00
ZonedDateTime newZdt = nyOdt.toZonedDateTime();
System.out.println(newZdt); //prints newzdt = 2024-05-01T06:00-04:00
```

Observe that there is no region information in the `ZonedDateTime` instance that was created from the `OffsetDateTime` instance. Since this `ZonedDateTime` instance does not have any region information, it can't apply rules for DST. This is proven by the following code:

```
System.out.println(nyZdt.plusMonths(7)); //prints 2024-12-01T06:00-
05:00[America/New_York]
System.out.println(newZdt.plusMonths(7)); //prints 2024-12-01T06:00-04:00
```

You can see that the the time zone offset for the new `ZonedDateTime` created from `nyZdt` changed from -4 to -5 (because DST ends on 3rd Nov 2024) upon adding 7 months to it but it did not change for `newZdt` because even though a `ZonedDateTime` knows how to handle DST, it can do so only if it is created with an official `ZoneId`.

Translating from `ZonedDateTime` and `OffsetDateTime` to `Instant`

Since an `Instant` corresponds to a time in UTC (which has 0 offset), translating a `ZonedDateTime` or a `OffsetDateTime` to an `Instant` is possible and just requires the offset amount of these types to be subtracted from the time that they contain. For example, a `ZonedDateTime` of `2024-12-01T06:00-05:00[America/New_York]` will translate to an `Instant` representing `2024-12-01T11:00Z` (because $6 - -5 = 11$). Similarly, a `ZonedDateTime` of `2024-04-29T19:30+05:30[Asia/Calcutta]` will translate to an `Instant` representing `2024-04-29T14:00:00Z` (because $19:30 - 5:30 = 14:00$).

However, an `Instant` cannot be translated to a `ZonedDateTime` or an `OffsetDateTime` because it wouldn't know what zone id or offset amount to use for the translation. Of course, you can create them from an `Instant` by providing a `ZoneId` or a `ZoneOffset`. For example: `instant.atZone(ZoneId.of("America/New_York"))` or `instant.atOffset(ZoneOffset.ofHoursMinutes(1, 20))`.

22.5 Working with Daylight savings time 🙌

Note: If you have no idea about Daylight Savings Time, it would be better if you read a few basic articles on it before reading this section. You may start with this Wikipedia page: https://en.wikipedia.org/wiki/Daylight_saving_time

In the previous section you saw that 10AM on 29th April 2024 in New York was

translated to 7.30PM in India. Normally, Eastern Time (ET) is 5 hours behind UTC but on 29th Apr 2024, Daylight Savings Time (DST) was in effect in ET (which means, the clocks had already been shifted 1 hour forward in New York) and so, ET was only 4 hours behind UTC on this date. India does not observe DST and so, Indian Standard Time (IST) is always 5.30 hours ahead of UTC. You may verify these numbers by printing `nyTime.getOffset()` and `indiaTime.getOffset()`. They will print `-04:00` and `+05:30` respectively. That is why the 10AM in ET was translated to 7.30PM in IST instead of 8.30PM on this day. This example shows that `ZonedDateTime` handles the adjustments required due to DST automatically.

Note that I have used ET and IST only for illustrating the logic. You do not need to remember the offsets for any time zone or the dates and the amount of shift required due to DST for any location.

In the real exam, the problem statement will include this information if it is needed to arrive at the correct answer.

Translating into gaps and overlaps created due to DST

The translation from ET to IST was easy, but what if, after the translation, the date and time at the target location doesn't exist? This could happen when the target location observes DST and the translated date/time happens to fall at the time when the clocks have been moved forward. For example, given that ET is 5 hours behind UTC and the DST starts on 10th March 2024 at 2AM (this means, as soon as the clocks in New York hit 2AM on this day, they are moved forward by 1 hour to show 3AM), what if you try to convert 10th March 2024 7.30AM UTC to ET? Normally, the time in ET would be 2.30AM but on 10th March 2024, 2.30AM doesn't exist in ET!

```
LocalDateTime ldt = LocalDateTime.of(2024, 03, 10, 7, 30, 0); //10th  
March 2024 7.30 AM
```

```
ZonedDateTime utcTime = ldt.atZone(ZoneId.of("Z")); //Z implies UTC  
ZonedDateTime nyTime =  
    utcTime.withZoneSameInstant(ZoneId.of("America/New_York"));  
System.out.println(nyTime);
```

The above code prints `2024-03-10T03:30-04:00[America/New_York]`, which shows that Java automatically moves the time **forward** to 3.30AM and the zone offset to -4 due to DST when the time at the destination falls into the gap.

The reverse happens when DST ends. The clocks are shifted back by 1 hour. So, for example, since DST ends at 2AM on 3rd Nov 2024 in New York, as soon the clocks hit this date/time, they are moved back to 1AM. Thus, people in New York get to live the hour between 1AM to 2AM twice on that day! During the first hour the Zone offset is -4 (because DST was in place at that time) and during the second hour, it is -5. This sounds cool but what happens when a time from another time zone that doesn't observe DST during this period is translated to New York time?

Let's work it out with the same two time zones - UTC and ET. Imagine that you are translating the time continuously from 4AM UTC onwards to ET. Translating any time before 5AM UTC to ET is easy. Since DST is "ON" in ET during that period, the time in ET would just be UTC minus 4 (instead of UTC minus 5). So, 5AM UTC would be 1AM ET, no doubt. During the time between 5AM UTC (inclusive) and 6AM UTC(exclusive), the DST is still ON because the clocks in ET haven't hit

2AM yet. Thus, the times between 5AM UTC (inclusive) and 6AM UTC (exclusive) map to 1AM ET (inclusive) to 2AM ET (exclusive) with an offset of **-4**. Now, the clocks in ET hit 2AM and the time in ET shifts back to 1AM. Therefore, 6AM UTC also maps to 1AM ET but with an offset of **-5**. Observe that two time periods in UTC (5AM to < 6AM and 6AM to < 7AM) map to the same time period in ET (1AM to < 2AM) **but with different offsets**. The following code bears this out:

```
ZoneId zUTC = ZoneId.of("GMT+0");
LocalDate ld = LocalDate.of(2024, 11, 3);
LocalTime lt1 = LocalTime.of(5, 0, 0);
LocalTime lt2 = LocalTime.of(6, 0, 0);
ZonedDateTime timeInUTC1 = ZonedDateTime.of(ld, lt1, zUTC);
ZonedDateTime timeInUTC2 = ZonedDateTime.of(ld, lt2, zUTC);
ZoneId zNY = ZoneId.of("America/New_York");
ZonedDateTime timeInNY1 = timeInUTC1.withZoneSameInstant(zNY);
ZonedDateTime timeInNY2 = timeInUTC2.withZoneSameInstant(zNY);
System.out.println(timeInNY1); //2024-11-03T01:00-
04:00[America/New_York]
System.out.println(timeInNY2); //2024-11-03T01:00-
05:00[America/New_York]
```

Observe that even though the time in both the cases is 1AM, the offsets are -4 and -5. Therefore, translating these date/times back to UTC will give you 5AM UTC and 6AM UTC respectively.

Another example:

```
//Given that DST starts on 2AM, 9th March 2025 and ends on 2AM,
2nd Nov 2025
ZoneId ny = ZoneId.of("America/New_York"); LocalTime lt =
LocalTime.of(1, 10);

ZonedDateTime zdt1 = ZonedDateTime.of(LocalDate.of(2025, 3, 9), lt, ny);
```



```
ZonedDateTime zdt2 = zdt1.plusHours(1);
System.out.println(zdt1.getHour()+ "/" + zdt1.getOffset()); //prints 1/-5
System.out.println(zdt2.getHour()+ "/" + zdt2.getOffset()); //prints 3/-4

ZonedDateTime zdt3 = ZonedDateTime.of(LocalDate.of(2025, 11, 2), lt, ny);
ZonedDateTime zdt4 = zdt3.plusHours(1);
System.out.println(zdt3.getHour()+ "/" + zdt3.getOffset()); //prints 1/-4
System.out.println(zdt4.getHour()+ "/" + zdt4.getOffset()); //prints 1/-5
```

22.6 Comparing date/time objects

One can compare date/time objects either using the instance methods of the date/time classes themselves or by finding out the difference between them using the `ChronoUnit` class.

22.6.1 Comparing date/time objects

There are four ways to compare date/time objects.

1. Using the `equals` method - All date/time classes override the `equals` method to compare the contents of the two objects. They only return `true` if the argument is an object of the same class and their contents match exactly.

```
LocalDate ld1 = LocalDate.now(); //assuming today is 2024-04-30
```

1.

```
LocalDate ld2 = LocalDate.parse("2024-04-30");
System.out.println(ld1.equals(ld2)); //prints true

LocalDateTime ltd = LocalDateTime.now();
System.out.println(ltd.equals(ld2)); //prints false
```

Recall that the signature of `equals` is `equals(Object obj)`, therefore, there is no compilation error when you pass an object of a different class to `equals`.

2. Using the `compareTo` method - All date/time classes implement `java.util.Comparable` interface, which has a `compareTo` method. Unlike the `equals` method, this method returns an `int`. If the date/time passed in the argument is later, this method returns a negative number, if the argument is exactly same, this method returns 0, and a positive number otherwise.

```
LocalDateTime ltd1 = LocalDateTime.now();//assuming today is 2024-04-30
LocalDateTime ltd2 = LocalDateTime.parse("2024-05-30T10:20");
System.out.println(ltd1.compareTo(ltd2)); //prints -1

LocalDateTime ltd3 = LocalDateTime.parse("2024-03-30T10:20");
System.out.println(ltd1.compareTo(ltd3)); //prints 1
```

One might expect that the return value would be the difference between the two dates, but the JavaDoc API description for this method does not make any such promise. Thus, you should rely only on the sign of the return value and not on the actual value.

3. Using the `isAfter/isBefore` methods - As their names suggest, these methods check if this date is after or before the date/time passed in the argument.

3. **LocalDate** ld1 = LocalDate.now();//assuming today is 2024-04-30
LocalDate ld2 = LocalDate.parse("2024-04-29");
System.out.println(ld1.isAfter(ld2)); //prints true
System.out.println(ld1.isBefore(ld2)); //prints false

Instant now = Instant.now();
Instant oneHrLater = now.plus(1, ChronoUnit.HOURS);
System.out.println(oneHrLater.isAfter(now)+" "+oneHrLater.isBefore(now));
//prints true false

The type of the argument must be the same as the type of the object on which these methods are called. So, for example, a statement such as `zonedDateTime.isAfter(localDateTime)` will not compile.

Determining the result of `isAfter` / `isBefore` is easy when the dates being compared are in the same time zone but can get very confusing when they are not. For example, can you tell what the following code will print?

```
LocalDateTime someDate = LocalDateTime.of(2025, 1, 1, 10, 0, 0);  
//1st Jan 2025 10 AM  
ZonedDateTime zJohn =  
someDate.atZone(ZoneId.of("America/New_York")); //zone offset is  
-5:00  
ZonedDateTime zJane = someDate.atZone(ZoneId.of("Asia/Calcutta"));  
//zone offset is +5:30  
System.out.println(zJohn.isAfter(zJane));
```

Don't feel bad if you don't get this. The way to do this is to first get the two dates on to a common time line, i.e., UTC. You saw how to do that in the previous section. `zJohn` translated to UTC is 2025/1/1 15:00 (10 - -5 = 15) and `zJane` translated to UTC is 2025/1/1 04:30 (10 - 5.30 = 4.30). It is now easy to see that `zJohn` is after `zJane` (because 15:00 is after 04:30) and so, the statement

3. will print `true` .

4. Using the `isEqual` method - This method is provided only by `LocalDate` and `LocalDateTime` . It checks if this date is equal to the date passed in the argument.

```
LocalDate ld1 = LocalDate.now();//assuming today is 2018-04-30
LocalDate ld2 = LocalDate.of(2018, 4, 29);
System.out.println(ld1.isEqual(ld2)); //prints false
System.out.println(ld1.isEqual(ld1)); //prints true
```

You may be wondering why we need the `isBefore/isAfter/isEqual` methods when we already have the `compareTo` and `equals` methods. Although this is beyond the scope of the exam, the difference between them is that `isBefore/isAfter/isEqual` methods do not take chronology, i.e., the calendar system into account while comparing, but `equals` and `compareTo` do. This difference becomes important when you compare a date from, say, a Thai calendar with a date from the regular ISO calendar.

`Period` has only three instance methods that deal with comparison - `equals` , `isZero` , and `isNegative` . `Equals` only returns true if all three components of the two `Periods` match. `IsZero` only returns `true` if all the three components of the `Period` are zero. `IsNegative` only returns `true` if any of the three components of the `Period` is less than zero. `Duration` has the same three methods too for comparison and they work the same way.

22.6.2 Using ChronoUnit to find the difference between two date/time objects 🙋

As mentioned before the various constants defined in the `ChronoUnit` enum

represent the different units of time. These constants can be used to find the difference between two date/time objects, which means it can be used to compare the two date/time objects albeit in a limited way. Here is an example:

```
LocalDateTime ld1 = LocalDateTime.now();//assuming it is 2024-04-27
16:43 right now
LocalDateTime ld2 = ld1.truncatedTo(ChronoUnit.DAYS);
System.out.println(ld1); //prints 2024-04-27T16:43:58.815418
System.out.println(ld2); //2024-04-27T00:00
System.out.println(ChronoUnit.MINUTES.between(ld1, ld2)); //prints -1003
System.out.println(ChronoUnit.HOURS.between(ld1, ld2)); //prints -16
System.out.println(ChronoUnit.DAYS.between(ld1, ld2)); //0
```

The output illustrates the following points about the `between` method:

1. The difference between two date time objects is computed in the same unit as the `ChronoUnit` constant on which the `between` method is invoked. That is why `ChronoUnits.MINUTES.between` converts the difference of 16 hours 43 minutes to 1003 minutes.
2. It returns "second argument" minus "first argument". That is why the the returned value is negative 1003.
3. The difference ignores/truncates the parts of the date/time that are less significant than the unit used. That is why the 58 seconds part was ignored from the calculation when the unit was `MINUTES` and the 43 minutes and 58 seconds part was ignored when the unit was `HOURS`.

The `between` method can be used to compare two different types of date/time objects but there is a catch. If the the types of the two objects are different, the second type is translated to an instance of the first type before the calculating the difference. Based on this information can you guess what will the following code print?

```
LocalDate ld = LocalDate.now();
```

```
LocalDateTime ldt = LocalDateTime.now();
System.out.println(ChronoUnit.DAYS.between(ld, ldt));
System.out.println(ChronoUnit.DAYS.between(ldt, ld));
```

The `LocalDateTime` object can be translated to a `LocalDate` by dropping the time component, which means both the `LocalDate` instances contain the same date and so, the first print statement prints 0. However, since a `LocalDate` doesn't have the time component, it can't be translated to a `LocalDateTime`. Therefore, the second print statement throws a `DateTimeException`. Ok, now how about the following?

```
LocalDateTime ldt = LocalDateTime.now();
ZonedDateTime zdt = ZonedDateTime.now();
System.out.println(ChronoUnit.DAYS.between(ldt, zdt));
System.out.println(ChronoUnit.DAYS.between(zdt, ldt));
```

Before answering, remember that `LocalDateTime` doesn't have zone information and so, it can't be converted into a `ZonedDateTime`. Therefore, the second print statement will throw an exception.

In the previous section you saw how a `ZonedDateTime` and an `OffsetDateTime` can be translated to an `Instant` but the reverse is not possible. Therefore, for example, `ChronoUnit.HOURS.between(Instant.now(), ZonedDateTime.now());` will work and return 0 but `ChronoUnit.HOURS.between(ZonedDateTime.now(), Instant.now());` will throw a `DateTimeException` at run time.

22.7 Quiz

Q 1. Given the following variable definitions:

```
LocalDateTime d1 = LocalDateTime.of(2024, 7, 1, 10, 30, 30);
```

```
LocalDateTime d2 = d1.plusMinutes(10);
Instant ins = Instant.now(); //assume that it is 10:30:30 AM UTC now

ZonedDateTime z1 = ins.atZone(ZoneId.of("America/New_York"));
ZonedDateTime z2 = ins.atZone(ZoneId.of("Asia/Calcutta"));

ZonedDateTime z3 = d1.atZone(ZoneId.of("America/New_York"));
ZonedDateTime z4 = d2.atZone(ZoneId.of("Asia/Calcutta"));
```

Assuming that America/New_York is 4 hours **behind** UTC and Asia/Calcutta is 5:30 hours **ahead** of UTC, determine the output generated by the following statements:

```
System.out.println(z1.isAfter(z2)+" "+z1.isBefore(z2));
System.out.println(z3.isAfter(z4));
System.out.println(d1.isEqual(d2.truncatedTo(ChronoUnit.MINUTES)));
```

Correct answer is:

```
false false
true
false
```

Calling `atZone(ZoneId )` on an `Instant` translates that `Instant` (which is nothing but a time in UTC) to a time in the given time zone. Since `z1` and `z2` are created using the same `Instant`, they represent the same time albeit in different time zones. Therefore, `z1.isEqual(z2)` will return `true` and `z1.isAfter(z2)` and `z1.isBefore(z2)` will return `false`.

Calling `atZone(ZoneId )` on a `LocalDateTime` doesn't translate the `LocalDateTime` (because a `LocalDateTime` is never specific to any region, so, there is nothing to translate from) but creates a `ZonedDateTime` by attaching the given time zone to the `LocalDateTime` object. Therefore, even though `z3` and `z4` are

created using the same `LocalDateTime`, they represent different times. To compare them, you need to translate these times into a common UTC time. Since NYC is 4 hours behind UTC, you have to add 4 to the NYC time to get the UTC time and since Calcutta is 5:30 hours ahead of UTC, you have to subtract 5:30 from the Calcutta time to get the UTC time. (Note that in terms of "offset" values, the offset for NYC is **-4** and the offset for Calcutta is **+5:30**). Thus, `z3` can be computed as 10:30+4, i.e., 14:30, and so `z3` is 2024/7/1 14:30Z and `z4` is 2024/7/1 05:00Z (10:30-5:30). Since 14:30 is **after** 5:00, `z3.isAfter(z4)` will return `true`.

`d1.plusMinutes(10)` produces a `LocalDateTime` of 24/7/1 10:40:30, which is assigned to `d2`. `d2.truncatedTo(ChronoUnit.MINUTES)` produces a `LocalDateTime` of 2024/7/1 10:40:00. This is not the same as the `d1` which contains 2024/7/1 10:30:30. Therefore, the last statement will print false.

Q 2. Given that Amsterdam is normally 1 hour ahead of UTC but observes Daylight savings time from 31 March 2024, 2AM to 27 October 2024, 03:00 AM, what will the following code print?

```
ZoneId zid = ZoneId.of("Europe/Amsterdam");
LocalDate ld = LocalDate.of(2024, 10, 27);
LocalTime lt = LocalTime.of(2, 30, 0);
ZonedDateTime amsterdamBefore = ZonedDateTime.of(ld, lt, zid);
ZonedDateTime amsterdamAfter = amsterdamBefore.plusHours(1);
System.out.println(amsterdamBefore.getOffset()+" "+amsterdamAfter.getOffset());
System.out.println(amsterdamBefore.getHour()+" "+amsterdamAfter.getHour());
System.out.println(amsterdamAfter.isAfter(amsterdamBefore));
```

Correct Answer is:

```
+02:00 +01:00
2 2
true
```


It is given that Amsterdam is normally 1 hour ahead of UTC. Therefore, when DST is **OFF**, its offset is +1 and when DST is **ON**, the offset is +2 (because DST moves the clocks forward). Since, on 27th March 2024, 2:30AM, DST is ON (according to the duration of DST given in the problem statement), `amsterdamBefore.getOffset()` will return +2 and `amsterdamAfter.getOffset()` will return +1.

It is given that DST ends at 3AM. This means the clocks are shifted backward by 1 hour at 3AM. Thus, adding 1 hour to 2:30AM will change the time to 3:30AM but then it will be shifted back to 2:30. Therefore, both `amsterdamBefore.getHour()` and `amsterdamAfter.getHour()` will return an int value of 2.

Observe that `amsterdamAfter` has been created by **adding** one hour to `amsterdamBefore`, which means `amsterdamAfter` is indeed "after" `amsterdamBefore`. Therefore, `amsterdamAfter.isAfter(amsterdamBefore)` will return `true`.

22.8 Converting Date/time objects to strings 📌

You can get a string representation of a date/time object using either the `toString` method or the `format(DateTimeFormatter dtf)` method.

Just like the single argument `parse` methods, the `toString` method uses the ISO format for generating the string. Here are a few examples:

```
System.out.println(LocalDate.now()); //prints 2024-04-27
System.out.println(LocalDateTime.now()); //prints 2024-04-27T14:34:46.425524900
System.out.println(OffsetDateTime.now()); //prints 2024-04-27T14:34:46.426840800+05:30
System.out.println(ZonedDateTime.now()); //prints 2024-04-27T14:34:46.426840800+05:30[Asia/Calcutta]
```

Observe that the output includes a character `'T'` that separates the date and the time components.

Here is an example that uses a custom `DateTimeFormatter` :

```
DateTimeFormatter dtf = DateTimeFormatter.ofPattern("yy MMM dd");  
System.out.println(dtf.format(LocalDate.now())); //prints 24 Apr 27  
System.out.println(dtf.format(LocalDateTime.now())); //prints 24 Apr 27
```

An interesting thing to note in the above output is that no exception is thrown at the second line. The pattern that we used to create the `DateTimeFormatter` does not include any information for time. Yet, when we used this formatter to format a `LocalDateTime`, it didn't complain. It simply ignored the time component of the `LocalDateTime` object. This is totally opposite to how the `parse` method behaves. If you try to parse a string that has a time component using this formatter, it will throw an exception.

```
LocalDate ld = LocalDate.parse("24 Apr 27 10:10", dtf);
```

The output from the above code is:

```
java.time.format.DateTimeParseException: Text '24 Apr 27 10:10' could not be  
parsed, unparsed text found at index 9
```

By the way, `DateTimeParseException` extends `DateTimeException` .

Passing a string as an argument to the `ofPattern` method is an example of "hardcoding". If you want to standardize the format in which you want to print dates across your application, using a hardcoded string is not a good idea because it is prone to typos and misunderstandings.

`DateTimeFormatter` provides a better way to create formatter objects through its static **ofXXX** methods. It has four such methods:

```
static DateTimeFormatter ofLocalizedDate(FormatStyle dateStyle)
static DateTimeFormatter ofLocalizedDateTime(FormatStyle
dateTimeStyle)
static DateTimeFormatter ofLocalizedDateTime(FormatStyle dateStyle,
FormatStyle timeStyle)
static DateTimeFormatter ofLocalizedTime(FormatStyle timeStyle)
```

The method that should be used depends on the kind of object you want to format. If you have a `LocalDate`, you should use `ofLocalizedDate`, if you have a `LocalTime`, you should use `ofLocalizedTime`, and if you have a `LocalDateTime` to format, you should use `ofLocalizedDateTime` to get the formatter. Using an incompatible formatter, such as using a formatter from `ofLocalizedTime` to format a `LocalDateTime`, will cause a `DateTimeException` to be thrown.

Observe that instead of accepting a string for a format, these methods accept an object of type `FormatStyle`. `FormatStyle` is actually an enum with four values - **FULL**, **LONG**, **MEDIUM**, and **SHORT**. You should use a combination of **ofXXX** method and `FormatStyle` to create a `DateTimeFormatter` that satisfies your needs. For example, the following code shows how you can print a date in full as well as in short form:

```
DateTimeFormatter dtf1 =
DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL);
DateTimeFormatter dtf2 =
DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT);
LocalDate ld = LocalDate.now();
System.out.println(dtf1.format(ld));//prints Saturday, April 27, 2024
System.out.println(dtf2.format(ld));//prints 4/27/24
```

Similarly, the following code prints a `LocalDateTime` in short and medium forms:

```
DateTimeFormatter dtf1 =  
DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);  
DateTimeFormatter dtf2 =  
DateTimeFormatter.ofLocalizedDateTime(FormatStyle.MEDIUM);  
  
LocalDateTime ld = LocalDateTime.now();  
  
System.out.println(dtf1.format(ld)); //prints 4/27/24 1:03 PM  
System.out.println(dtf2.format(ld)); //prints Apr 27, 2024 1:03:59 PM  
//(Remember that the exact output depends on your default locale)
```

Using `FormatStyle.LONG` or `FormatStyle.FULL` for formatting a `LocalDateTime` or `LocalTime` will throw an exception at run time because these forms expect time zone information in the time component, which is not present in `LocalDateTime` or `LocalTime`.

The exam **does not** expect you to remember the formats in which these formatters print the date/time. You only need to know how standardized formatters can be created using the `ofXXX` methods mentioned above instead of the `ofPattern` methods.

Note that `Period` and `Duration` do not have any format method. To convert them into a string, you have to use their `toString` method. This method generates a string in the same ISO-8601 format that I talked about before. Here is an example:

```
Period p = Period.parse("p13m5w");  
System.out.println(p); //prints P13M35D  
Duration d = Duration.parse("pt26h66m3s");  
System.out.println(d); //prints PT27H6M3S
```

Observe that 5 weeks have been converted into 35 days but 13 months have remained unchanged. Also, 66 minutes in the given duration string have been converted to 1 hour and 6 minutes.

22.9 Exercise

1. Print current date and time of your machine and print what it is at this moment at UTC/GMT.
2. Given that two users from two different time zones created their accounts on 1st May 2024 2.30PM in their time zone. Write code that determines which user created their account before the other.
3. Given that a report was created by an application on 1st May 2024 2.30PM in the time zone of your application server. Write code that prints this time for users located in different time zones.